

Version 2.0 | Updated — Free Stack Config

CHANGED SECTION: Project Structure — Config Files

```
src/config/
■■■ database.ts      # Prisma client → Neon.tech
■■■ redis.ts         # Upstash Redis (replaces ElasticCache)
■■■ cloudflare.ts    # R2 storage (replaces aws.ts)
■■■ email.ts         # Resend SDK (replaces SES/Nodeemailer)
■■■ app.ts           # Express app setup (unchanged)
```

CHANGED SECTION: Environment Variables

KodetoCareer — Document 09 (Updated Free Stack) — Confidential

[illegible]

Removed env vars (all were paid services):

- `AWS_REGION`
- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`
- `S3_BUCKET_RAW`
- `S3_BUCKET_PROCESSED`
- `CLOUDFRONT_DOMAIN`
- `MEDIACONVERT_ENDPOINT`
- `MEDIACONVERT_ROLE_ARN`
- `SES_REGION`
- `MSG91_API_KEY`
- `MSG91_SENDER_ID`

CHANGED SECTION: Config File Implementations

src/config/cloudflare.ts (replaces aws.ts)

```
import { S3Client } from '@aws-sdk/client-s3';

// Cloudflare R2 is S3-compatible – use the same SDK
export const r2Client = new S3Client({
  region: 'auto',
  endpoint: `https://${process.env.CF_ACCOUNT_ID}.r2.cloudflarestorage.com`,
  credentials: {
    accessKeyId: process.env.R2_ACCESS_KEY_ID!,
    secretAccessKey: process.env.R2_SECRET_ACCESS_KEY!,
  },
});

export const R2_BUCKET = process.env.R2_BUCKET_NAME!;
```

```
export const CDN_BASE_URL = process.env.R2_PUBLIC_URL!;
```

```
// Helper: get public CDN URL for a file key
export const getCdnUrl = (fileKey: string): string => {
  return `${CDN_BASE_URL}/${fileKey}`;
};
```

src/config/redis.ts (updated for Upstash)

```
import { Redis } from '@upstash/redis';
import IORedis from 'ioredis';

// @upstash/redis – for direct cache operations (API response caching, sessions)
export const redis = new Redis({
  url: process.env.UPSTASH_REDIS_REST_URL!,
  token: process.env.UPSTASH_REDIS_REST_TOKEN!,
});

// ioredis connection – for Bull queue workers
export const redisForBull = new IORedis(process.env.UPSTASH_REDIS_URL!, {
  maxRetriesPerRequest: null, // Required for Bull
  enableReadyCheck: false,    // Required for Upstash
  tls: {},                    // Upstash requires TLS
});
```

src/config/email.ts (updated — Resend replaces SES)

```
import { Resend } from 'resend';

const resend = new Resend(process.env.RESEND_API_KEY!);

interface SendEmailOptions {
  to: string | string[];
  subject: string;
  html: string;
  replyTo?: string;
}

export const sendEmail = async ({ to, subject, html, replyTo }: SendEmailOptions) => {
  try {
    const result = await resend.emails.send({
      from: process.env.FROM_EMAIL!,
      to: Array.isArray(to) ? to : [to],
      subject,
      html,
      replyTo,
    });
    return { success: true, id: result.data?.id };
  } catch (error) {
    console.error('Email send failed:', error);
    return { success: false, error };
  }
};

// Batch send (for announcements to multiple students)
// Resend supports up to 100 recipients per call
export const sendBulkEmail = async (
  recipients: Array<{ email: string; name: string }>,
  subject: string,
```

```

    htmlTemplate: (name: string) => string
  ) => {
    const BATCH_SIZE = 50; // Stay well under Resend limits
    const batches = [];

    for (let i = 0; i < recipients.length; i += BATCH_SIZE) {
      batches.push(recipients.slice(i, i + BATCH_SIZE));
    }

    for (const batch of batches) {
      await Promise.all(
        batch.map(r => sendEmail({
          to: r.email,
          subject,
          html: htmlTemplate(r.name),
        }))
      );
      // Rate limit: pause 1 second between batches to respect 100 emails/day limit
      if (batches.length > 1) await new Promise(r => setTimeout(r, 1000));
    }
  };
};

```

src/services/storage.service.ts (updated for R2)

```

import {
  PutObjectCommand,
  GetObjectCommand,
  DeleteObjectCommand
} from '@aws-sdk/client-s3';
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';
import { r2Client, R2_BUCKET, getCdnUrl } from '../config/cloudflare';

// Generate presigned URL for direct client upload to R2
export const getPresignedUploadUrl = async (
  fileKey: string,
  contentType: string,
  expiresIn: number = 3600
): Promise<string> => {
  const command = new PutObjectCommand({
    Bucket: R2_BUCKET,
    Key: fileKey,
    ContentType: contentType,
  });
  return getSignedUrl(r2Client, command, { expiresIn });
};

// Get a temporary presigned URL for private file access
// NOTE: For public files (videos, notes), just use getCdnUrl() directly – no signing...
export const getPresignedDownloadUrl = async (
  fileKey: string,
  expiresIn: number = 3600
): Promise<string> => {
  const command = new GetObjectCommand({
    Bucket: R2_BUCKET,
    Key: fileKey,
  });
  return getSignedUrl(r2Client, command, { expiresIn });
};

// Upload buffer directly (used by certificate generator)

```

```
export const uploadBuffer = async (
  fileKey: string,
  buffer: Buffer,
  contentType: string
): Promise<string> => {
  await r2Client.send(new PutObjectCommand({
    Bucket: R2_BUCKET,
    Key: fileKey,
    Body: buffer,
    ContentType: contentType,
  }));
  return getCdnUrl(fileKey);
};

// Delete a file
export const deleteFile = async (fileKey: string): Promise<void> => {
  await r2Client.send(new DeleteObjectCommand({
    Bucket: R2_BUCKET,
    Key: fileKey,
  }));
};
```

CHANGED SECTION: SMS Service — Replaced with Email Alert

REMOVED: src/services/sms.service.ts (MSG91 integration)

REPLACED WITH: Email-based alerts only. All OTPs and critical alerts go via `sendEmail()` from `email.service.ts`.

If SMS is needed in the future (after revenue generation), add this:

```
// src/services/sms.service.ts — ADD ONLY WHEN REVENUE EXISTS
// Using Twilio (free trial: 15 USD credits, then $0.0075/SMS)
import twilio from 'twilio';

const client = twilio(process.env.TWILIO_SID, process.env.TWILIO_AUTH_TOKEN);

export const sendSMS = async (to: string, message: string) => {
  return client.messages.create({
    body: message,
    from: process.env.TWILIO_PHONE_NUMBER,
    to: `+91${to}`,
  });
};
```

CHANGED SECTION: Developer Quickstart (Updated)

```
# Prerequisites
node --version    # v20+
npm --version     # v10+
# No local PostgreSQL needed — Neon.tech connection string used
# No local Redis needed — Upstash connection string used

# Clone and install
git clone https://github.com/kodetocareer/api.git
```

```
cd kodetocareer-api
npm install

# Environment
cp .env.example .env
# Fill in:
# DATABASE_URL → from Neon.tech dashboard
# UPSTASH_REDIS_REST_URL + TOKEN → from Upstash dashboard
# UPSTASH_REDIS_URL → from Upstash dashboard
# CF_ACCOUNT_ID + R2 keys → from Cloudflare dashboard
# RESEND_API_KEY → from Resend dashboard
# FCM keys → from Firebase console
# JWT keys → generate with openssl (see .env.example comments)

# Database (runs migrations on Neon.tech)
npx prisma migrate dev
npx prisma db seed

# Run development server
npm run dev
# API runs on http://localhost:3000

# Run tests
npm test

# Local development note:
# Since we use Neon (serverless PostgreSQL) and Upstash (serverless Redis),
# no Docker Compose is needed for databases.
# Just Node.js installed locally is sufficient.
```

Docker Compose (optional, for fully offline local dev):

```
# docker-compose.yml – only if you want local DB/Redis for offline dev
version: '3.8'
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: kodetocareer
      POSTGRES_PASSWORD: localpassword
    ports:
      - "5432:5432"
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

CHANGED SECTION: Deployment Quickstart

```
# On fresh Oracle Cloud Ubuntu 22.04 ARM instance

# Install Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Install PM2 and Nginx
sudo npm install -g pm2
sudo apt-get install -y nginx
```

```
# Clone and setup
cd /app
git clone https://github.com/kodetocareer/api.git
cd api
npm ci --production
cp .env.example .env
# ... fill in production values

# Database migration
npx prisma migrate deploy

# Build
npm run build

# Start with PM2
pm2 start ecosystem.config.js --env production
pm2 save
pm2 startup # Auto-start on server reboot

# Nginx config
sudo nano /etc/nginx/sites-available/kodetocareer
# (add reverse proxy config to port 3000)
sudo ln -s /etc/nginx/sites-available/kodetocareer /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
# SSL handled by Cloudflare – no Certbot needed
```

End of Updated Development Specification Sections

All other sections (project structure, code standards, API spec, certificate spec, CSV import, analytics calcs, git workflow, testing) remain identical to Dev Specification v1.